

★ Member-only story

本地部署 Gemma 4 並實作模型量化：從 Hugging Face 到 Ollama 的完整流程



Allenah · 9 min read · Apr 28, 2026



近一年來，愈來愈多開發者開始嘗試將大型語言模型部署到本地端。一方面是因為本地推理能降低長期 API 成本，另一方面則是出於更實際的需求：希望自行掌握模型、資料與執行環境，讓模型不只是「可呼叫的服務」，而是真正可被調校、測試與整合的基礎設施。

這篇文章記錄的是一次完整的本地部署流程。我使用的模型是 **Gemma 4 26B A4B Instruct**，從 Hugging Face 下載原始權重開始，接著轉換為 GGUF 格式、執行量化，最後透過 Ollama 建立本地可執行模型。

[Open in app](#) ↗



Medium



Search



Write



為什麼需要進行量化？

很多人第一次接觸本地 LLM 時，最常見的想法是：「我已經下載到模型了，為什麼不能直接丟進 Ollama 跑？」

原因在於**模型檔案格式**、**推理引擎格式**與**部署框架格式**並不相同。

以 Hugging Face 上下載到的模型為例，它通常是給 Transformers 生態系使用的，權重會以 safetensors 儲存，搭配 config.json、tokenizer.json 等檔案。這種格式非常適合訓練、微調與 Python 推理，但並不是 llama.cpp 與 Ollama 最直接使用的格式。若你想讓模型在本地端以較低資源執行，通常會走向 GGUF 與量化這條路。

所以，整個流程其實是在處理四件不同的事：

Hugging Face 權重下載

→ 轉換為 llama.cpp 可讀的 GGUF

→ 進行量化，降低模型資源需求

→ 匯入 Ollama，建立推理用 runtime

我的GPU是NVIDIA GeForce RTX 4060 Ti，VRAM 只有16G，以這樣的硬體條件來看，如果不經過量化，直接下載並執行 **Gemma 4 26B A4B Instruct** 幾乎不可行。

26B 等級的模型，本身參數量就非常大，若維持原始較高精度格式，例如 FP16，模型權重本體就會占用非常可觀的記憶體空間；而在實際推理時，除了模型權重之外，還需要額外預留 KV cache、runtime buffer，以及框架本身的載入開銷。也就是說，模型並不是「剛好塞進 16GB VRAM」就能順利執行，而是通常還需要更多餘裕，才能支撐穩定推理。

Parameters	BF16 (16-bit)	SFP8 (8-bit)	Q4_0 (4-bit)
Gemma 4 E2B	9.6 GB	4.6 GB	3.2 GB
Gemma 4 E4B	15 GB	7.5 GB	5 GB
Gemma 4 31B	58.3 GB	30.4 GB	17.4 GB
Gemma 4 26B A4B	48 GB	25 GB	15.6 GB

Table 1. Approximate GPU or TPU memory required to load Gemma 4 models based on parameter count and quantization level.

根據 Google 官方網站的 Gemma 4 記憶體需求表，Gemma 4 26B A4B Instruct 在 BF16 / 16-bit 精度下，推理所需的 GPU 記憶體大約是 48 GB；即使降到 SFP8 / 8-bit，也仍然需要約 25 GB，而 Q4_0 / 4-bit 則約為 15.6 GB。這也說明了，對只有 16GB VRAM 的 RTX 4060 Ti 而言，若不經過量化，26B 模型幾乎不可能完整載入並穩定執行；即使到了 4-bit 量化，也已經非常接近顯存上限，因此量化對我來說不是單純的優化手段，而是讓這顆模型真正有機會在本地端落地的前提。

模型來源：從 Hugging Face 下載 Gemma 4

這次使用的模型為：

```
google/gemma-4-26B-A4B-it
```

連結: <https://huggingface.co/google/gemma-4-26B-A4B-it>

Step 1：安裝 Hugging Face CLI

先確認你有 Python，然後安裝：

```
pip install huggingface_hub
```

```
sudo apt install -y git build-essential cmake python3 python3-pip
```

```
pip3 install transformers sentencepiece safetensors accelerate
```

Step 2：登入 Hugging Face

```
hf auth login
```

貼你的 token（去 HuggingFace 網站拿）

Access Tokens				
User Access Tokens				+ Create new token
Access tokens authenticate your identity to the Hugging Face Hub and allow applications to perform actions based on token permissions. Do not share your Access Tokens with anyone ; we regularly check for leaked Access Tokens and remove them immediately.				
Name	Value	Last Refreshed Date	Last Used Date	Permissions
ollama-download	hf_...ECZm	1 minute ago	-	READ

Step 3：下載模型

完成這些準備後，即可使用以下指令下載模型：

```
hf download google/gemma-4-26B-A4B-it
```

下載完成後，資料夾大致會長成這樣：

```
gemma4-f16/  
├── config.json  
├── generation_config.json  
├── tokenizer.json  
├── tokenizer_config.json  
├── model-00001-of-000xx.safetensors  
├── model-00002-of-000xx.safetensors  
└── ...
```

這裡最重要的是那些 `.safetensors` 檔案。它們就是模型權重本身，而其餘的 `config` 與 `tokenizer` 則負責描述模型結構與文字切分方式。

下載完成後，拿到的是什麼？

這一步的成果，並不是「可以直接丟到 Ollama 執行的模型」，而是**原始權重**。更準確地說，你拿到的是一份適合 Transformers 生態系使用的 checkpoint。

在這個階段，模型通常是以 FP16（16-bit floating point）的精度儲存。FP16 的優點是保留相對高的數值精度，對模型表現比較友善；缺點則是檔案很大、推理時對記憶體與顯存的需求也很高。

以 26B 這種等級的模型來說，如果你完全保留原始精度，對一般本地工作站並不算輕量。這也是為什麼後面一定會進入 GGUF 與 quantization 這兩個步驟。

為什麼還要轉成 GGUF？

Hugging Face 的模型格式適合 Python 與 Transformers 使用，但如果你要把模型交給 llama.cpp 或 Ollama，最常見的做法是先轉為 GGUF。

GGUF 可以視為一種針對本地推理優化的模型封裝格式。它不是單純把副檔名改掉而已，而是會把模型權重、metadata、tokenizer 資訊等整理成 llama.cpp 生態系容易存取與載入的形式。

Step 1：下載 llama.cpp

我們可以直接拉 llama.cpp repo 至本地：

```
git clone https://github.com/ggerganov/llama.cpp
cd llama.cpp
```

Step 2：編譯

```
cmake -B build
cmake --build build -j
```

Step 3：轉 GGUF

這一步通常使用 llama.cpp 提供的轉換腳本來完成：

```
python convert_hf_to_gguf.py ./gemma4-f16 \
  --outfile gemma4.gguf \
  --outtype f16
```

這裡的意思是：把 ./gemma4-f16 裡的 Hugging Face 模型，轉成一個名為 gemma4.gguf 的 GGUF 檔案，並保留 f16 精度。

這裡有一個很重要的觀念：**轉換格式不等於量化**。

- **格式轉換**：處理相容性，讓模型能被新的推理框架讀取
- **量化**：處理精度與大小，讓模型更容易在有限資源下執行

量化是什麼？為什麼它是本地部署的關鍵？

量化（Quantization）可以用一句話概括：

將模型權重從較高精度的數值表示，轉換為較低精度的表示，以換取更小的模型體積與更低的推理成本。

若用比較直觀的方式理解，原本模型權重可能以 FP16 或 FP32 儲存，每個數值都保留比較細緻的小數表達能力；量化之後，這些權重會被壓縮到 8-bit、4-bit，甚至更低。數值變得沒那麼精細，但模型會變得更容易在消費級 GPU 或較有限的 RAM 環境中執行。

這是一種典型的 **precision-efficiency trade-off**。

你犧牲一部分精度，換取可部署性與推理效率。

在大型語言模型的實務中，量化不是冷門技巧，而是一個非常核心的工程手段。因為如果沒有量化，很多模型根本無法離開雲端資源，在一般本地設備上實際落地。

開始量化：將 GGUF 轉為 Q4

在這次實作中，我先做的是 Q4 量化。對應指令如下：

```
./build/bin/llama-quantize gemma4.gguf gemma4-q4.gguf q4_K_M
```

這行指令可以拆成三個部分理解：

1. `gemma4.gguf`：原始 GGUF 模型
2. `gemma4-q4.gguf`：量化後輸出的模型檔名
3. `q4_K_M`：指定量化方法

這裡的 `q4_K_M` 並不是隨機命名，而是 llama.cpp 常見的一種量化策略。簡單理解即可：

- `q4` 表示 4-bit quantization
- `K` 表示一種 block-wise 的量化形式
- `M` 通常代表 mixed / balanced 的變體，兼顧大小與效果

它之所以常被拿來當作入門實作的第一選擇，是因為它通常能在**模型大小**、**推理速度**與**品質損失**之間取得相對平衡。

建立 Modelfile：讓 Ollama 知道怎麼讀你的模型

當你已經有量化完成的 GGUF 檔案，下一步就是把它交給 Ollama。這裡 Ollama 不會直接讀資料夾裡的任意檔案，而是透過一個叫做 **Modelfile** 的設定檔，描述模型來源與推理參數。

建立方式如下：

```
nano Modelfile
```

內容可以先從最基本版本開始：

```
FROM ./gemma4-q4.gguf
```

告訴 Ollama 這個模型是由目前資料夾下的 `gemma4-q8.gguf` 建立而來。

如果你想順便把推理參數一起寫進去，也可以在 Modelfile 中加入：

```
FROM ./gemma4-q4.gguf

PARAMETER temperature 0.7
PARAMETER top_p 0.9
PARAMETER num_ctx 4096
```

這幾個參數分別控制：

- temperature：輸出的隨機性
- top_p：nucleus sampling 範圍
- num_ctx：上下文視窗大小

從工程角度來看，Modelfile 可以視為 Ollama 的模型建置設定。它不是模型本體，而是模型的 runtime 描述方式。

用 Ollama 建立本地模型

當 Modelfile 準備好後，就可以正式把模型交給 Ollama：

```
ollama create gemma4-q4 -f Modelfile
```

這一步相當重要，因為它代表你不是只把檔案放在某個資料夾裡，而是正式在 Ollama 裡註冊了一個可執行模型。

到這裡為止，從 Hugging Face 原始 checkpoint 到 Ollama 本地模型的鏈路就完成了。

用ollama list 來看看

NAME	ID	SIZE	MODIFIED
gemma4-q4:latest	3f1e6c87a736	16 GB	4 hours ago
gemma4:e4b	c6eb396dbd59	9.6 GB	4 days ago

q4 只佔了16GB

看看GPU 使用狀況:

NVIDIA-SMI 560.35.02				Driver Version: 560.94			CUDA Version: 12.6		
GPU	Name			Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr.	ECC
Fan	Temp	Perf			Memory-Usage		GPU-Util	Compute M.	
				Pwr:Usage/Cap				MIG M.	
0	NVIDIA GeForce RTX 4060 Ti			On	00000000:01:00.0	On			
0%	41C	P8	9W / 165W		15439MiB / 16380MiB			13%	Default
								N/A	
								N/A	

只用16G VRAM 就可以跑Gemma 26B模型囉!

- Gemma 4
- Ollama
- Hugging Face
- Model Quantization



Written by Allenah

174 followers · 36 following

Edit profile

AI Engineer | Data Science | Writing, Sharing, Learning

